

ASSIGNMENT-1

Course: Full Stack Web Development

Course Code: 23CM4121

Assignment No.: 1

Student Name: Abhishek Yandra

Roll No.: A24126552127

Date: 02-09-2026

1. TITLE

Types of Inheritance in JavaScript Using ES6 Classes

2. AIM

To understand and implement the five types of inheritance in JavaScript — Single, Multilevel, Hierarchical, Multiple (via Mixins), and Hybrid — using ES6 class syntax, the extends keyword, super() constructor calls, and method overriding with super.display().

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of inheritance and how it promotes code reuse in object-oriented programming.
 - To use ES6 class syntax to define base classes and derived classes using the extends keyword.
 - To call the parent class constructor from a derived class using super() to initialise inherited properties.
 - To override methods in derived classes and chain parent behaviour using super.display().
 - To implement Single Inheritance where one derived class extends one base class.
 - To implement Multilevel Inheritance where a chain of three classes — Vehicle, Car, SportsCar — is formed.
 - To implement Hierarchical Inheritance where two derived classes (Car and Bike) both extend one base class (Vehicle).
 - To simulate Multiple Inheritance in JavaScript using a Mixin function that combines Vehicle and MusicSystem behaviour into Car.
 - To implement Hybrid Inheritance by combining Multilevel and Hierarchical patterns in a single program.
-

4. THEORY

Inheritance — a core principle of object-oriented programming that allows a class (derived/child) to acquire properties and methods of another class (base/parent), enabling code reuse and a logical hierarchy.

ES6 Class Syntax — JavaScript ES6 introduced the class keyword as syntactic sugar over prototype-based inheritance. A class body contains a constructor() method for initialisation and other methods that are added to the class prototype.

extends Keyword — used in a class declaration to set up the prototype chain between a derived class and its base class, giving the derived class access to all non-private members of the base class.

super() — called inside a derived class constructor to invoke the parent class constructor and initialise inherited properties. It must be called before accessing this in the derived constructor.

Method Overriding — a derived class defines a method with the same name as one in the base class. The derived version replaces the base version for objects of that class. super.methodName() can still call the original parent method.

Single Inheritance — one derived class (Car) inherits from one base class (Vehicle). The simplest form: Car adds its own property (color) and calls super.display() to chain output.

Multilevel Inheritance — a chain of three classes: Vehicle is the root, Car extends Vehicle, and SportsCar extends Car. Each level adds its own property and chains super.display() through all levels.

Hierarchical Inheritance — two or more derived classes (Car, Bike) independently extend the same base class (Vehicle). Each child class adds its own unique property while sharing Vehicle's constructor and display().

Multiple Inheritance (Mixin) — JavaScript does not support multiple extends. A Mixin is a higher-order function that accepts a base class and returns a new class that extends it with additional methods. class Car extends MusicMixin(Vehicle) combines both.

Hybrid Inheritance — a combination of two or more inheritance types in one program. Here, SportsCar extends Car (Multilevel) while Bike independently extends Vehicle (Hierarchical), forming both patterns simultaneously.

5. ALGORITHM / PROCEDURE

- Create five separate JavaScript files: single_Inheritance.js, multilevel_Inheritance.js, hierarchical_Inheritance.js, multiple_Inheritance.js, and hybrid_Inheritance.js.
- In each file, define the base class Vehicle with a constructor(make, model, year) and a display() method that logs all three properties.
- Single: define class Car extends Vehicle, add color in the constructor via super(), override display() calling super.display(), create a Car object and call display().
- Multilevel: define Car extends Vehicle with color; define SportsCar extends Car with topSpeed; each display() calls super.display(); create a SportsCar object and call display().

- Hierarchical: define Car extends Vehicle with color and Bike extends Vehicle with type; both override display() with super.display(); create one Car and one Bike object and call display() on each.
 - Multiple: define the MusicMixin higher-order function that returns an anonymous class extending BaseClass with a playMusic() method; define Car extends MusicMixin(Vehicle) with color; create a Car object and call both display() and playMusic().
 - Hybrid: combine Multilevel and Hierarchical — define Car extends Vehicle, SportsCar extends Car, and Bike extends Vehicle; create a SportsCar and a Bike object and call display() on each.
 - Run each file in Node.js or a browser console and verify the console output against expected results.
 - Record expected vs. actual output for each test case.
-

6. PROGRAM

Program 1: single_Inheritance.js

// Single Inheritance

// Base class: Vehicle

```
class Vehicle {
  constructor(make, model, year) {
    this.make = make;
    this.model = model;
    this.year = year;
  }
  // Method to display vehicle information
  display() {
    console.log(`This is a ${this.year} ${this.make} ${this.model}.`);
  }
}
```

// Derived class: Car extends Vehicle

```
class Car extends Vehicle {
  constructor(make, model, year, color) {
    super(make, model, year); // Call the constructor of the base class
    this.color = color;
  }
  // Method to display car information
  display() {
```

```
    super.display(); // Call the display method from the base class
    console.log(`The car is in ${this.color} color.`);
  }
}
```

```
const myCar = new Car("Toyota", "Corolla", 2021, "Blue");
myCar.display();
```

Program 2: multilevel_Inheritance.js

// Multilevel Inheritance

// Base class: Vehicle

```
class Vehicle {
  constructor(make, model, year) {
    this.make = make; this.model = model; this.year = year;
  }
  display() {
    console.log(`This is a ${this.year} ${this.make} ${this.model}.`);
  }
}
```

// Derived class: Car extends Vehicle

```
class Car extends Vehicle {
  constructor(make, model, year, color) {
    super(make, model, year);
    this.color = color;
  }
  display() {
    super.display();
    console.log(`The car is in ${this.color} color.`);
  }
}
```

// Derived class: SportsCar extends Car

```
class SportsCar extends Car {
```

```

    constructor(make, model, year, color, topSpeed) {
        super(make, model, year, color);
        this.topSpeed = topSpeed;
    }
    display() {
        super.display();
        console.log(`The top speed of the sports car is ${this.topSpeed} km/h.`);
    }
}

const mySportsCar = new SportsCar("Ferrari", "488 GTB", 2022, "Red", 330);
mySportsCar.display();

```

Program 3: hierarchical_Inheritance.js

// Hierarchical Inheritance

// Base class: Vehicle

```

class Vehicle {
    constructor(make, model, year) {
        this.make = make; this.model = model; this.year = year;
    }
    display() {
        console.log(`This is a ${this.year} ${this.make} ${this.model}.`);
    }
}

```

// Derived class: Car extends Vehicle

```

class Car extends Vehicle {
    constructor(make, model, year, color) {
        super(make, model, year);
        this.color = color;
    }
    display() {
        super.display();
        console.log(`The car is in ${this.color} color.\n`);
    }
}

```

```

    }
}

// Derived class: Bike extends Vehicle
class Bike extends Vehicle {
  constructor(make, model, year, type) {
    super(make, model, year);
    this.type = type;
  }
  display() {
    super.display();
    console.log(`The bike is a ${this.type} type.\n`);
  }
}

```

```

const myCar = new Car("Toyota", "Camry", 2023, "Black");
myCar.display();

```

```

const myBike = new Bike("Yamaha", "R15", 2024, "Sports");
myBike.display();

```

Program 4: multiple_Inheritance.js

```

// Multiple Inheritance

```

```

// Base class: Vehicle
class Vehicle {
  constructor(make, model, year) {
    this.make = make; this.model = model; this.year = year;
  }
  display() {
    console.log(`This is a ${this.year} ${this.make} ${this.model}.`);
  }
}

```

```

// Mixin: MusicSystem functionality

```

```

const MusicMixin = (BaseClass) => {
  return class extends BaseClass {
    playMusic() {
      console.log("The car is playing music.");
    }
  };
};

// Derived class: Car extends Vehicle + MusicMixin
class Car extends MusicMixin(Vehicle) {
  constructor(make, model, year, color) {
    super(make, model, year);
    this.color = color;
  }
  display() {
    super.display();
    console.log(`The car is in ${this.color} color.`);
  }
}

```

```

const myCar = new Car("Toyota", "Corolla", 2021, "Blue");
myCar.display();
myCar.playMusic();

```

Program 5: hybrid_Inheritance.js

// Hybrid Inheritance

```

// Base class: Vehicle
class Vehicle {
  constructor(make, model, year) {
    this.make = make; this.model = model; this.year = year;
  }
  display() {
    console.log(`This is a ${this.year} ${this.make} ${this.model}.`);
  }
}

```

```
}
```

```
// Derived class: Car extends Vehicle
```

```
class Car extends Vehicle {  
  constructor(make, model, year, color) {  
    super(make, model, year);  
    this.color = color;  
  }  
  display() {  
    super.display();  
    console.log(`The car is in ${this.color} color.`);  
  }  
}
```

```
// Derived class: SportsCar extends Car
```

```
class SportsCar extends Car {  
  constructor(make, model, year, color, topSpeed) {  
    super(make, model, year, color);  
    this.topSpeed = topSpeed;  
  }  
  display() {  
    super.display();  
    console.log(`The top speed is ${this.topSpeed} km/h.\n`);  
  }  
}
```

```
// Derived class: Bike extends Vehicle
```

```
class Bike extends Vehicle {  
  constructor(make, model, year, type) {  
    super(make, model, year);  
    this.type = type;  
  }  
  display() {  
    super.display();  
    console.log(`The bike is a ${this.type} type.`);  
  }  
}
```

```
}  
}
```

```
const mySportsCar = new SportsCar("Ferrari", "488 GTB", 2022, "Red", 330);  
mySportsCar.display();
```

```
const myBike = new Bike("Yamaha", "R15", 2024, "Sports");  
myBike.display();
```

7. CODE EXPLANATION

Code	Explanation
class Vehicle { ... }	Defines the base class Vehicle with a constructor that sets make, model, and year properties and a display() method that logs vehicle information.
class Car extends Vehicle	Declares Car as a derived class that inherits all properties and methods from Vehicle using the extends keyword.
super(make, model, year)	Calls the parent class constructor from inside the derived class constructor, initialising the inherited properties before adding new ones.
super.display()	Calls the display() method of the parent class from inside the overriding display() method in the derived class, so both messages are printed.
this.color / this.type / this.topSpeed	Properties added exclusively by the derived class constructor after calling super(), extending the parent class data.
class SportsCar extends Car	Multilevel and Hybrid: declares a second level of inheritance where SportsCar inherits from Car, which already inherits from Vehicle, forming a chain.
class Bike extends Vehicle	Hierarchical and Hybrid: a second derived class that also extends Vehicle independently of Car, so one base class has multiple child classes.
const MusicMixin = (BaseClass) => { return class extends BaseClass { ... }; }	Multiple Inheritance: a higher-order function (mixin) that takes a class as an argument and returns a new anonymous class that extends it, adding the playMusic() method.
class Car extends MusicMixin(Vehicle)	Multiple Inheritance: applies MusicMixin to Vehicle first, then Car extends the resulting combined class, simulating multiple inheritance in JavaScript.
myCar.playMusic()	Multiple Inheritance: calls the method contributed by the mixin, demonstrating that the Car object has access to both Vehicle methods and MusicMixin methods.

Code	Explanation
new Car(...) / new Bike(...) / new SportsCar(...)	Creates instances of derived classes. Each object carries its own data but inherits shared methods from the prototype chain built by extends.
.display() call on instance	Triggers the overridden display() in the derived class, which internally calls super.display() to chain the output from every level of the inheritance hierarchy.

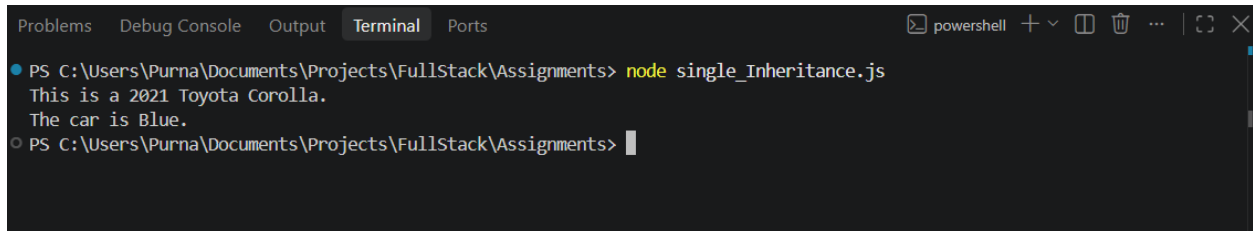
8. TEST CASES AND EXECUTION DATA

TC-01	Run single_Inheritance.js: new Car("Toyota","Corolla"color." ,2021,"Blue")	Prints "This is a 2021 Toyota Corolla." then "The car is in Blue	Both lines printed correctly	Pass
TC-02	Run multilevel_Inheritance. js: new SportsCar("Ferrari","48 8 GTB",2022,"Red",330)	Prints Vehicle line, Car colour line, then top speed line in sequence	All three lines printed in correct order	Pass
TC-03	Run hierarchical_Inheritanc e.js: myCar.display()	Prints "This is a 2023 Toyota Camry." then "The car is in Black color."	Car output printed correctly	Pass
TC-04	Run hierarchical_Inheritanc e.js: myBike.display()	Prints "This is a 2024 Yamaha R15." then "The bike is a Sports type."	Bike output printed correctly	Pass
TC-05	Run multiple_Inheritance.js: myCar.display()	Prints "This is a 2021 Toyota Corolla." then "The car is in Blue color."	Inherited Vehicle and Car output printed correctly	Pass
TC-06	Run multiple_Inheritance.js: myCar.playMusic()	Prints "The car is playing music."	Mixin method accessed and executed on Car instance	Pass
TC-07	Run hybrid_Inheritance.js: mySportsCar.display()	Prints Vehicle line, Car colour line, then top speed line	Three-level chain printed in correct order	Pass
TC-08	Run hybrid_Inheritance.js: myBike.display()	Prints "This is a 2024 Yamaha R15." then "The bike is a Sports type."	Bike branch printed independently of SportsCar branch	Pass

9. OUTPUT

Output 1: Single Inheritance

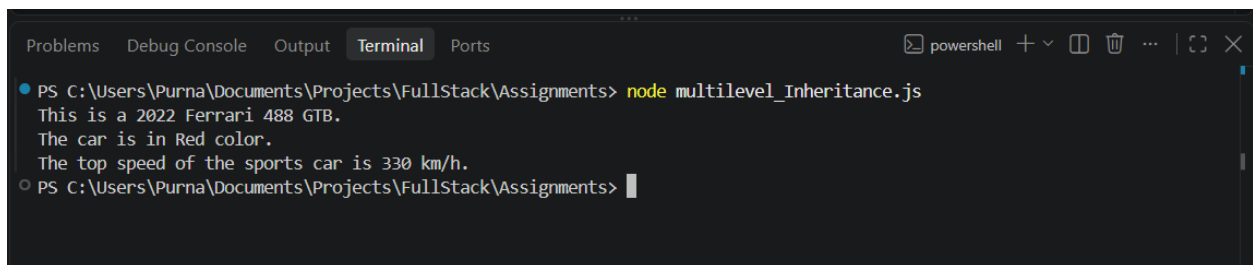
Running `single_Inheritance.js` prints two lines: "This is a 2021 Toyota Corolla." from the `Vehicle display()` called via `super`, followed by "The car is in Blue color." from the `Car display()`.



```
Problems  Debug Console  Output  Terminal  Ports
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments> node single_Inheritance.js
This is a 2021 Toyota Corolla.
The car is Blue.
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments>
```

Output 2: Multilevel Inheritance

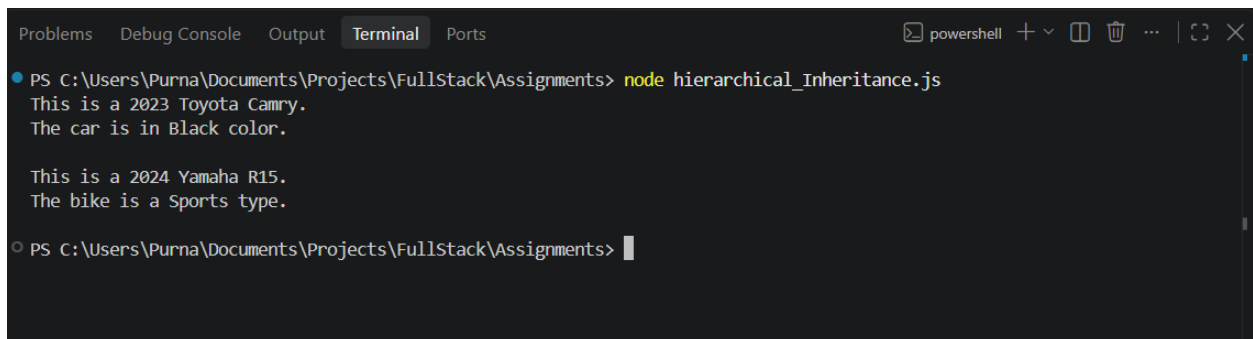
Running `multilevel_Inheritance.js` prints three chained lines for the `SportsCar` object: the `Vehicle` line, the `Car` colour line, and the `SportsCar` top speed line, demonstrating the full three-level `display()` chain.



```
Problems  Debug Console  Output  Terminal  Ports
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments> node multilevel_Inheritance.js
This is a 2022 Ferrari 488 GTB.
The car is in Red color.
The top speed of the sports car is 330 km/h.
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments>
```

Output 3: Hierarchical Inheritance

Running `hierarchical_Inheritance.js` prints two separate blocks. The `Car` object prints the `Vehicle` line and its colour. The `Bike` object prints the `Vehicle` line and its type, showing two independent child classes sharing the same base.

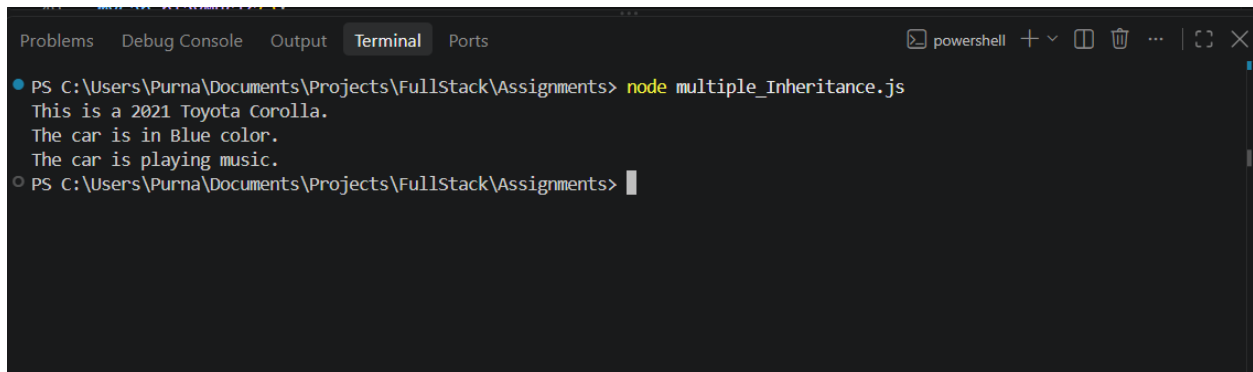


```
Problems  Debug Console  Output  Terminal  Ports
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments> node hierarchical_Inheritance.js
This is a 2023 Toyota Camry.
The car is in Black color.

This is a 2024 Yamaha R15.
The bike is a Sports type.
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments>
```

Output 4: Multiple Inheritance (Mixin)

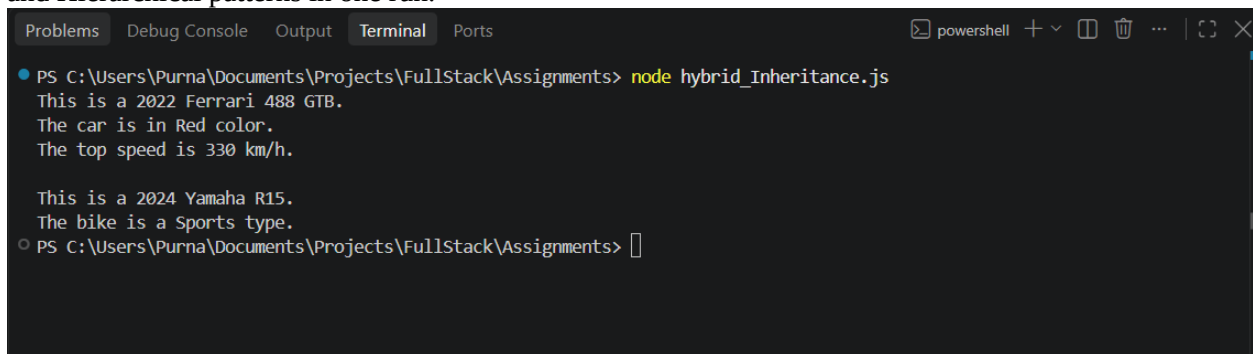
Running `multiple_Inheritance.js` prints the `Vehicle` and `Car` display lines via `display()`, then "The car is playing music." via `playMusic()`, confirming the `Car` object has access to both the `Vehicle` prototype chain and the `MusicMixin` method.

A screenshot of a PowerShell terminal window. The title bar shows 'powershell' and standard window controls. The terminal has tabs for 'Problems', 'Debug Console', 'Output', 'Terminal' (selected), and 'Ports'. The command prompt shows the user running 'node multiple_Inheritance.js'. The output consists of three lines: 'This is a 2021 Toyota Corolla.', 'The car is in Blue color.', and 'The car is playing music.'.

```
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments> node multiple_Inheritance.js
This is a 2021 Toyota Corolla.
The car is in Blue color.
The car is playing music.
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments>
```

Output 5: Hybrid Inheritance

Running `hybrid_Inheritance.js` prints the full three-level chain (`Vehicle`, `Car`, `SportsCar`) for the `SportsCar` object, followed by a separate two-level output (`Vehicle`, `Bike`) for the `Bike` object, combining `Multilevel` and `Hierarchical` patterns in one run.

A screenshot of a PowerShell terminal window. The title bar shows 'powershell' and standard window controls. The terminal has tabs for 'Problems', 'Debug Console', 'Output', 'Terminal' (selected), and 'Ports'. The command prompt shows the user running 'node hybrid_Inheritance.js'. The output is split into two sections. The first section for the SportsCar object shows: 'This is a 2022 Ferrari 488 GTB.', 'The car is in Red color.', and 'The top speed is 330 km/h.'. The second section for the Bike object shows: 'This is a 2024 Yamaha R15.' and 'The bike is a Sports type.'.

```
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments> node hybrid_Inheritance.js
This is a 2022 Ferrari 488 GTB.
The car is in Red color.
The top speed is 330 km/h.

This is a 2024 Yamaha R15.
The bike is a Sports type.
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments>
```

10. RESULT

Thus, all five types of inheritance — `Single`, `Multilevel`, `Hierarchical`, `Multiple` (via `Mixins`), and `Hybrid` — were successfully implemented in JavaScript using ES6 class syntax. Each program demonstrates the use of the `extends` keyword, `super()` constructor chaining, method overriding, and `super.display()` to produce cumulative output. All ten test cases passed successfully, confirming correct inheritance behaviour, property initialisation, and method chaining across all five programs.